FastAPI Reference

Dependencies - `Depends()` and `Security()`

`Depends()`

Dependencies are handled mainly with the special function `Depends()` that takes a callable.

Here is the reference for it and its parameters.

You can import it directly from `fastapi`:

```
from fastapi import Depends
```

`func` fastapi.Depends

```
Depends(dependency=None, *, use_cache=True, scope=None)
```

Declare a FastAPI dependency.

It takes a single "dependable" callable (like a function).

Don't call it directly, FastAPI will call it for you.

Read more about it in the FastAPI docs for Dependencies.

Example

```
from typing import Annotated

from fastapi import Depends, FastAPI

app = FastAPI()


async def common_parameters(q: str | None = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}


@app.get("/items/")
async def read_items(commons: Annotated[dict, Depends(common_parameters)]):
    return commons
```

PARAMETER	DESCRIPTION
`dependency`	A "dependable" callable (like a function). Don't call it directly, FastAPI will call it for you, just pass the object directly. Read more about it in the FastAPI docs for Dependencies **TYPE:** `Callable[..., Any] | None` **DEFAULT:** `None`
`use_cache`	By default, after a dependency is called the first time in a request, if the dependency is declare for the rest of the request (for example if the dependency is needed by several dependencies),

Ask AI

PARAMETER	DESCRIPTION
	value will be re-used for the rest of the request. Set `use_cache` to `False` to disable this behavior and ensure the dependency is called again (if declared more than once) in the same request. Read more about it in the FastAPI docs about sub-dependencies **TYPE:** `bool` **DEFAULT:** `True`
`scope`	Mainly for dependencies with `yield`, define when the dependency function should start (the code before `yield`) and when it should end (the code after `yield`). • `"function"`: start the dependency before the *path operation function* that handles the request, end the dependency after the *path operation function* ends, but **before** the response is sent back to the client. So, the dependency function will be executed **around** the *path operation* ***function***. • `"request"`: start the dependency before the *path operation function* that handles the request (similar to when using `"function"`), but end **after** the response is sent back to the client. So, the dependency function will be executed **around** the **request** and response cycle. Read more about it in the FastAPI docs for FastAPI Dependencies with yield **TYPE:** `Literal['function', 'request'] | None` **DEFAULT:** `None`

Source code in `fastapi/param_functions.py`

```
def Depends(  # noqa: N802
    dependency: Annotated[
        Callable[..., Any] | None,
        Doc(
            """
            A "dependable" callable (like a function).

            Don't call it directly, FastAPI will call it for you, just pass the object
            directly.

            Read more about it in the
            [FastAPI docs for Dependencies](https://fastapi.tiangolo.com/tutorial/dependencies/)
            """
        ),
    ] = None,
    *,
    use_cache: Annotated[
        bool,
        Doc(
            """
            By default, after a dependency is called the first time in a request, if
            the dependency is declared again for the rest of the request (for example
            if the dependency is needed by several dependencies), the value will be
            re-used for the rest of the request.

            Set `use_cache` to `False` to disable this behavior and ensure the
            dependency is called again (if declared more than once) in the same request.

            Read more about it in the
            [FastAPI docs about sub-dependencies]
(https://fastapi.tiangolo.com/tutorial/dependencies/sub-dependencies/#using-the-same-dependency-
multiple-times)
            """
        ),
    ] = True,
    scope: Annotated[
        Literal["function", "request"] | None,
        Doc(
            """
            Mainly for dependencies with `yield`, define when the dependency function
            should start (the code before `yield`) and when it should end (the code
            after `yield`).

            * `"function"`: start the dependency before the *path operation function*
                that handles the request, end the dependency after the *path operation
                function* ends, but **before** the response is sent back to the client.
                So, the dependency function will be executed **around** the *path operation
                **function***.
            * `"request"`: start the dependency before the *path operation function*
                that handles the request (similar to when using `"function"`), but end
                **after** the response is sent back to the client. So, the dependency
                function will be executed **around** the **request** and response cycle.

            Read more about it in the
            [FastAPI docs for FastAPI Dependencies with yield]
(https://fastapi.tiangolo.com/tutorial/dependencies/dependencies-with-yield/#early-exit-and-
scope)
            """
        ),
    ] = None,
) -> Any:
    """
    Declare a FastAPI dependency.

    It takes a single "dependable" callable (like a function).

    Don't call it directly, FastAPI will call it for you.

    Read more about it in the
    [FastAPI docs for Dependencies](https://fastapi.tiangolo.com/tutorial/dependencies/).

    **Example**
```

```
    ```python
    from typing import Annotated

    from fastapi import Depends, FastAPI

    app = FastAPI()


    async def common_parameters(q: str | None = None, skip: int = 0, limit: int = 100):
        return {"q": q, "skip": skip, "limit": limit}


    @app.get("/items/")
    async def read_items(commons: Annotated[dict, Depends(common_parameters)]):
        return commons
    ```
    """
    return params.Depends(dependency=dependency, use_cache=use_cache, scope=scope)
```

`Security()`

For many scenarios, you can handle security (authorization, authentication, etc.) with dependencies, using `Depends()`.

But when you want to also declare OAuth 2 scopes, you can use `Security()` instead of `Depends()`.

You can import `Security()` directly from `fastapi`:

```
from fastapi import Security
```

`func` fastapi.Security

```
Security(dependency=None, *, scopes=None, use_cache=True)
```

Declare a FastAPI Security dependency.

The only difference with a regular dependency is that it can declare OAuth 2 scopes that will be integrated with OpenAPI and the automatic UI docs (by default at `/docs`).

It takes a single "dependable" callable (like a function).

Don't call it directly, FastAPI will call it for you.

Read more about it in the FastAPI docs for Security and in the FastAPI docs for OAuth 2 scopes.

Example

```
from typing import Annotated

from fastapi import Security, FastAPI

from .db import User
from .security import get_current_active_user

app = FastAPI()

@app.get("/users/me/items/")
async def read_own_items(
    current_user: Annotated[User, Security(get_current_active_user, scopes=["items"])]
):
    return [{"item_id": "Foo", "owner": current_user.username}]
```

PARAMETER	DESCRIPTION
`dependency`	A "dependable" callable (like a function). Don't call it directly, FastAPI will call it for you, just pass the object directly. Read more about it in the FastAPI docs for Dependencies **TYPE:** `Callable[..., Any] | None` **DEFAULT:** `None`
`scopes`	OAuth 2 scopes required for the *path operation* that uses this Security dependency. The term "scope" comes from the OAuth 2 specification, it seems to be intentionally vague and interpretable. It normally refers to permissions, in cases to roles. These scopes are integrated with OpenAPI (and the API docs at `/docs`). So they are visible in the OpenAPI specification. Read more about it in the FastAPI docs about OAuth 2 scopes **TYPE:** `Sequence[str] | None` **DEFAULT:** `None`
`use_cache`	By default, after a dependency is called the first time in a request, if the dependency is declared again for the rest of the request (for example if the dependency is needed by several dependencies), the value will be re-used for the rest of the request. Set `use_cache` to `False` to disable this behavior and ensure the dependency is called again (if declared more than once) in the same request. Read more about it in the FastAPI docs about sub-dependencies **TYPE:** `bool` **DEFAULT:** `True`

Source code in `fastapi/param_functions.py`

```python
def Security(  # noqa: N802
    dependency: Annotated[
        Callable[..., Any] | None,
        Doc(
            """
            A "dependable" callable (like a function).

            Don't call it directly, FastAPI will call it for you, just pass the object
            directly.

            Read more about it in the
            [FastAPI docs for Dependencies](https://fastapi.tiangolo.com/tutorial/dependencies/)
            """
        ),
    ] = None,
    *,
    scopes: Annotated[
        Sequence[str] | None,
        Doc(
            """
            OAuth2 scopes required for the *path operation* that uses this Security
            dependency.

            The term "scope" comes from the OAuth2 specification, it seems to be
            intentionally vague and interpretable. It normally refers to permissions,
            in cases to roles.

            These scopes are integrated with OpenAPI (and the API docs at `/docs`).
            So they are visible in the OpenAPI specification.

            Read more about it in the
            [FastAPI docs about OAuth2 scopes]
(https://fastapi.tiangolo.com/advanced/security/oauth2-scopes/)
            """
        ),
    ] = None,
    use_cache: Annotated[
        bool,
        Doc(
            """
            By default, after a dependency is called the first time in a request, if
            the dependency is declared again for the rest of the request (for example
            if the dependency is needed by several dependencies), the value will be
            re-used for the rest of the request.

            Set `use_cache` to `False` to disable this behavior and ensure the
            dependency is called again (if declared more than once) in the same request.

            Read more about it in the
            [FastAPI docs about sub-dependencies]
(https://fastapi.tiangolo.com/tutorial/dependencies/sub-dependencies/#using-the-same-dependency-
multiple-times)
            """
        ),
    ] = True,
) -> Any:
    """
    Declare a FastAPI Security dependency.

    The only difference with a regular dependency is that it can declare OAuth2
    scopes that will be integrated with OpenAPI and the automatic UI docs (by default
    at `/docs`).

    It takes a single "dependable" callable (like a function).

    Don't call it directly, FastAPI will call it for you.

    Read more about it in the
    [FastAPI docs for Security](https://fastapi.tiangolo.com/tutorial/security/) and
    in the
    [FastAPI docs for OAuth2 scopes](https://fastapi.tiangolo.com/advanced/security/oauth2-
scopes/).
```

```
    **Example**

    ```python
    from typing import Annotated

    from fastapi import Security, FastAPI

    from .db import User
    from .security import get_current_active_user

    app = FastAPI()

    @app.get("/users/me/items/")
    async def read_own_items(
        current_user: Annotated[User, Security(get_current_active_user, scopes=["items"])]
    ):
        return [{"item_id": "Foo", "owner": current_user.username}]
    ```
    """
    return params.Security(dependency=dependency, scopes=scopes, use_cache=use_cache)
```